Metadata Convention

Status: ✓ Adopted during Stage 2 **Applies to:** Click-to-Source 3D, v0.1 and forward
Companion documents: Click-to-Source Final Approach, Click-to-Source Technical Approaches, Click-to-Source Tech Stack, Stage 1 Experimental Report

---

Purpose

This document defines the single canonical way provenance metadata is structured and attached to runtime objects in Click-to-Source 3D. Every future component of the project — the overlay UI, the resolution engine, the MCP tool surface, and eventually the AST auto-instrumentation — is expected to agree with this contract rather than invent its own.

The convention below is not a proposal. Where it makes a claim, that claim is backed by the Stage 1 Experimental Report (7 experiments, raw runtime logs, UUID/identity comparisons). Where a question remains genuinely open, it is documented as open rather than guessed at.

---

1. The `SourceRef` Shape

```ts
type SourceRef = {
  file: string;
  function: string;
  line: number;
  args: Record<string, unknown>;
  schemaVersion?: number;
}
```

This shape was exercised, unmodified, across every Stage 1 experiment: single objects, multiple generated instances, ordinary React re-renders, explicit mesh recreation, memoized generation, and genuine Vite HMR. It held in every case. It is locked as the v0.1 contract.

`schemaVersion` **(optional, unused in v0.1):** reserved for the point where Stage 6.5's auto-instrumentation transform may generate `SourceRef` records with a different shape or provenance strategy than manual tagging does. Adding it now costs nothing; retrofitting it later would mean touching every consumer of `SourceRef`.

---

2. Canonical Tagging Rule

> **Provenance must be declared through the JSX `userData` prop as part of the mesh's declarative definition — e.g. `<mesh userData={{ sourceRef }}>`. Provenance must never be attached imperatively after mount (`mesh.userData.sourceRef = ...` inside a `useEffect`, ref callback, or event handler).**

Why this rule, specifically

Stage 1 (Experiments 4 and 7) established that a React Three Fiber mesh has two independent lifecycles that do not always move together:

- **The generation-data lifecycle**, governed by whatever produces the mesh's props (a plain function call, or a `useMemo`-cached result).
- **The runtime mesh lifecycle**, governed by React's reconciler — which keys and recreates (or reuses) the underlying `THREE.Mesh` based on JSX element type and position/key, independent of whether the data feeding its props changed.

Declaring `userData` as a prop means provenance is re-applied on every render, regardless of which lifecycle actually changed underneath it — so it is correct whether the mesh was reused (props merely updated) or recreated (a fresh mesh received fresh props). An imperative one-time assignment after mount has no way to know which case it's in, and drifts out of sync the moment either lifecycle fires without the other.

What this rule does not yet solve

Stage 1 tested this rule against cheap, effectively-instant generation (cubes). It did not test it against generation slow enough that a click could land between an old mesh's stale `userData` and a new render's updated `userData` finishing. This is a real gap, addressed below.

---

3. Deferred: Stale-Provenance Timing Window

Policy: v0.1 accepts the possibility of transient stale provenance during expensive regeneration. This was not observed in Stage 1 — cube generation was too fast to expose it — and will be re-evaluated during Stage 4 dogfooding, where generation (terrain, trees) is slow enough for the window to plausibly matter.

This is a deliberate deferral, not an oversight. It is written down here so that revisiting it later is a planned check, not a surprised rediscovery.

---

4. Future Auto-Instrumentation (Design Discussion — Not Decided)

v0.1 relies entirely on manual tagging under the rule in Section 2. A future stage (6.5, per the Tech Stack roadmap) will replace manual tagging with compile-time instrumentation. That instrumentation needs a way to know which functions to instrument — an **opt-in strategy**.

Three candidate strategies have been identified:

Strategy	Advantages	Disadvantages
Directory include (config-based)	Zero developer effort once configured	Instruments by *location*, not *intent* — can't distinguish a generator function from an unrelated helper living in the same folder
Comment annotation (`// @click-to-source`)	Explicit, attached to the function itself	"Soft" explicit — comments are non-semantic, ignored by refactor tooling, and can go stale without breaking anything
Wrapper API (`clickToSource(createTree)`)	Explicit, semantic, discoverable, survives refactors	Adds visible syntax; requires developer adoption of an API surface

No strategy is selected in Stage 2. Selection is deferred until Stage 6.5, when the instrumentation pipeline is actually implemented and there is real experience to decide from — not before.

Selection Criteria (for the future decision, not the decision itself)

When a strategy is chosen, it should be evaluated against the following. These are not weighted equally — the first two are treated as harder constraints, since accurate provenance is the project's core differentiator; the rest are desirable but not individually disqualifying:

Harder constraints:

- Low false positives (doesn't instrument things it shouldn't)
- Low false negatives (doesn't miss things it should instrument)

Softer, desirable properties:

- Minimal manual developer effort
- Explicit, legible developer intent
- Refactor-friendly (survives file moves, renames)
- IDE discoverable
- Zero runtime overhead after compilation — *this assumes whichever syntax is chosen at the source level is fully compiled away by the AST transform; it is not a claim that, e.g., the wrapper syntax must be zero-cost as written today, only that the final instrumented output should be.*

The goal of writing these down now is that Stage 6.5 asks "which candidate satisfies the criteria established in Stage 2," not "what do we feel like implementing today."

Non-Goals

This specification does **not** define:

- AST instrumentation implementation
- Source maps
- Overlay UI design
- Code editing / write-back mechanics
- Runtime synchronization between edits and the live scene
- MCP tool schemas
- Inspector panel behavior

Those belong to their respective stages. Keeping them out of this document is intentional — this file exists to answer exactly one question ("how is provenance structured and attached") and nothing else, so it stays a stable reference rather than accumulating implementation detail that will be revisited and rewritten stage by stage.

Summary

After Stage 2, there is no ambiguity about:

- what a `SourceRef` is,
- where it lives (`userData` prop, declaratively),
- how it's attached (JSX, every render, never imperative post-mount),
- when it's attached (at creation *and* at every subsequent render, by construction of the rule above),
- what this convention intentionally does not yet solve (stale-timing window, auto-instrumentation strategy).

Stages 3 and beyond build on this contract. They do not renegotiate it.